SYSTEM SECURITY ASSESSMENT

Assess systems the way you build them.

A fully AI-enabled assessment program, humans included.

BRUCE POTTER · JAWNCON 0X03

00 — DISCLAIMER

Don’t believe anything I say.

Test it against your own systems. That’s kind of the whole point of this talk.

Attackers just went breadth-first.

Humans pick a target and go deep. Agents try every mistake and misconfig, everywhere, and do it again tomorrow.

> SO
>
> That changes what “secure enough” means.



Both sides of the equation just moved.

01

Attacks go broad

Cheap, tireless agents hunting for any mistake, everywhere, all the time.

02

Code goes fast

AI-assisted development is building and shipping code at a pace we've never seen.

A point-in-time assessment is stale before the report ships.

Our system assessments are still shaped like a human attacker.

	AI-driven attacker	Typical system assessment
Shape	Breadth-first, everything	Depth-first, a scoped slice
Cadence	Continuous	Once a year, maybe
Cost per try	Close to zero	A team for weeks
New code	Reads it as it ships	Saw what existed last spring

The attacker covers the whole system. Do we?

Broad, repeatable system assessment is now a requirement for building systems.

01

Not a luxury for big clients

02

Not an annual pentest

03

AI does the work. Humans own the judgment.

Broad system assessment used to be a niche tool.

THEN: BUILD A LAB

For when failure isn't tolerable

- Stand up a lab around one product or platform
- Full breadth, plus deep focus where defects are security critical
- One big, expensive, point-in-time verdict

NOW: MAKE IT REPEATABLE

For every system you build

- Same breadth, same rigor, same method
- Light checks on every change, full runs when it matters
- Largely automated with AI. Humans keep the judgment calls.

The method works. It just never scaled past the lab. That's what we're going to fix.

It isn't a pentest.

THE PENTESTER HEARS

"Break into the system."

THE SYSTEM OWNER HEARS

"Tell me how secure my system is."

A pentest finds point-in-time vulnerabilities. Useful! But it doesn't tell you how secure the system is, and it's stale by the next deploy.

AI code review: necessary, not sufficient.

KEEP IT IN THE PIPELINE

- Modern LLMs are really good at finding code defects
- Runs on every PR, at the speed code ships
- Keeps up with AI-written code
- Way past what old static analysis could do

WHAT IT CAN'T SEE

- Design flaws: a broken protocol, perfectly coded
- How components behave together
- Deployment, config, and operations
- Whether the requirements are even right

It's one stage of the program, not the program.

We've been poking at security for a long time.

1851

Alfred Hobbs picks the "unpickable" locks

Opens the Bramah and Chubb locks at the Great Exhibition, then argues that talking openly about weaknesses helps honest people more than crooks.

1883

Kerckhoffs writes down the rule

A system should stay secure even if the enemy knows everything about it except the key. Design gets judged in the open.

Then the computers showed up.

1970

Ware Report

Computer security is a whole-system problem: hardware, software, people, procedures.

1972

Anderson Report

Air Force planning study. Gives us the reference monitor and makes the case for penetration testing.

1974

Karger & Schell

Tiger team on Multics. Finds design and code holes, and sketches the trojaned compiler a decade before Thompson.

Then the systems got too big to get your arms around.

THEN

Self-contained, self-hosted

- Your hardware, your data center, a few vendors
- Changed in releases, months or years apart
- A small team could read the whole design

NOW

Cloud and third parties

- Cloud platforms, SaaS, and third-party APIs
- Much of the risk lives with third parties
- Changes constantly, much of it written by AI

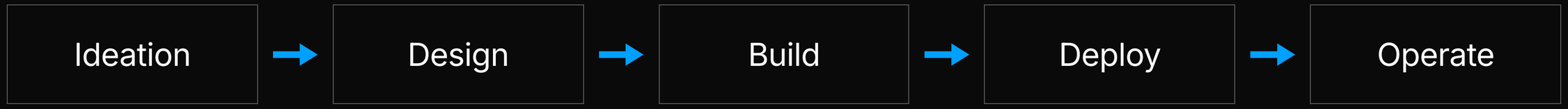
Assessment cost grew with complexity: more time, more effort, more expertise. So broad assessment retreated to the niche cases, the lab and the system that couldn’t fail.

03 — WHERE THAT LEFT US

Real system assessments became rare.

So did the people who can do them. AI changes that math.

Security is an emergent property.



IT EMERGES AT EVERY STAGE

Every decision adds or removes some: a design trade, a library pick, a deploy flag, an ops shortcut.

IT GETS BOLTED ON AT EVERY STAGE

A late design review, a WAF in front of the build, a scanner after deploy, a runbook after the incident.

You can’t assess one stage and call it done. Assess the whole system.

Security is one of the -ilities.

Scalability	Availability	Performance	Reliability
Maintainability	Usability	Quality	**Security**

Non-functional requirements we expect every system to have. Treat security as one component of overall quality. *(Thanks, Cigital.)*

The -ilities fight each other.

Security vs. performance

Encrypt and inspect everything, and you pay for it in latency.

Security vs. availability

The auth service is down. Fail closed and lock everyone out, or fail open?

Security vs. usability

Add friction and users build workarounds you never assessed.

Security vs. maintainability

Controls nobody understands get misconfigured, then switched off.

Assess security alone and you miss the trade the team made to hit its latency target. Assess it in context.

Security you can see, and security you can't.

FUNCTIONAL · YOU CAN SEE IT

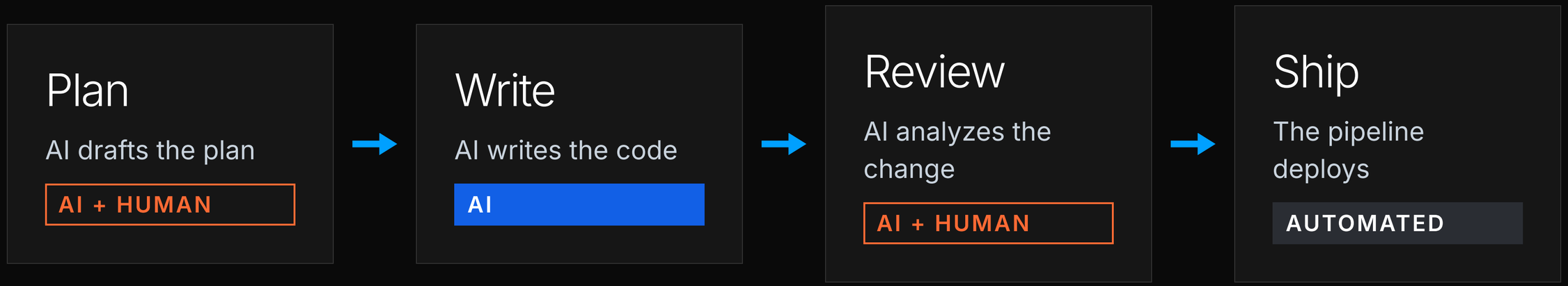
- Passwords and authentication
- Encryption
- Roles and entitlements
- Access control

NON-FUNCTIONAL · YOU CAN'T

- No race conditions
- No side-channel leaks
- No exploitable code defects
- Resilient in the face of poor configuration
- Holds up under abuse

A system assessment has to cover both. Breadth-first attackers live in the right one.

We already build software this way.



So why do we still assess our systems by hand, with targeted, depth-focused assessments once a year?

So assess systems the same way.

01	02	03	04	05	06	07
Ingest	**Analyze**	**Plan tests**	**Write tests**	**Run in CI/CD**	**Report**	**Compare**
System description in	Functional and NFR analysis	Prioritized test list	Code, harnesses, playbooks	Light per PR; full runs on big changes or a schedule	Met and not met	Diff vs. past reports
AI	AI + HUMAN	AI + HUMAN	AI + HUMAN	AI	AI	AI + HUMAN

Then start over when the system changes materially, or enough time has passed.

05 — THE SHIFT

AI does the work. Humans own the judgment.

We’re at the point where AI can help drive broad assessment in a repeatable, scalable way. What follows is the process I’ve used to build assessment labs, reworked to run with AI inside the development process.

Four human gates: the requirements, the test plan, the tests, and the verdict.

01

Start with a description of the system.

AI DOES

Reads design docs, diagrams, API specs, IaC, and code

Builds the model: components, data flows, trust boundaries, identities

Lists the third parties we trust, and for what

Flags gaps: “the docs say X, the code says Y”

> HUMAN · LIGHT TOUCH
>
> Hand over the real description, not the brochure
>
> Spot-check the model against reality. Everything downstream depends on it.

Output → a system model, versioned in the repo next to the code

02

Turn the system into requirements.

AI DOES

- Drafts functional requirements: authn, crypto, roles
- Drafts non-functional ones: no races, no leaks, holds under abuse
- Maps hotspots: trust boundaries, parsers, unenforced assumptions
- Seeds from CWE, ASVS, and design-flaw catalogs

> **HUMAN GATE**
>
> Decide what “secure” means for this system
>
> Strike the noise. Add what only experience knows.
>
> Sign off on the hotspot map

Output → a numbered requirements list and a hotspot map

03

Brainstorm the tests, then rank them.

AI DOES

- Brainstorms tests per requirement, design-level and code-level
- Ranks by impact, test cost, similarity, and coverage
- Groups tests that share setup
- Numbers them in a hierarchy: 3.2.1

HUMAN GATE

- Approve the priorities
- Aim the deep, expensive tests at the critical spots
- Say what's out of scope, out loud

Output → a prioritized test plan: intent, description, expected result

Describe tests so they can fail.

Test element	A performance test	A security test
Trigger	1,000 requests/sec arrive	Attacker replays a captured session token
Conditions	Normal ops, peak hour	From the internet, after the user logs out
Expected behavior	Every request is served	Token rejected, event logged
Pass / fail	p99 latency under 200 ms	100% rejected, alert within 5 min

Write security tests the way teams already write performance tests. If you can’t say what failure looks like, it isn’t a test yet.

04

Write the tests, then test the tests.

AI DOES

- Writes test code, harnesses, and fixtures
- Writes agent playbooks for the fuzzy, breadth-first stuff
- Ties every test back to its requirement ID

HUMAN GATE

Does each test actually test its intent?

Plant the bug and make sure the test catches it

Kill tests that pass no matter what

Output → a validated security test suite, in the repo next to the code

05

Run it in the pipeline, at the right cadence.

PER PR · LIGHT

AI code review plus tests touching the changed areas

MATERIAL CHANGE

Full run. If the design moved, rerun 01–03 too.

ON A SCHEDULE

Full run, daily or weekly, with the agent sweep.

NOT EVERY BUILD

Full runs are heavyweight. Spend them where they count.

HUMAN · NOTHING ROUTINE

Failures route to the owning team, like any broken build.

Output → fresh results when it matters, not once a year

06

Write the report on every run.

AI DOES

Requirements met, and not met

Coverage stated test by test, including what didn’t run

Findings in context: what they touch, what else stands in the way

Design flaws and code bugs called out separately

> HUMAN · NOT YET
>
> The report is evidence, not a verdict
>
> One report is a snapshot. The value shows up when you compare them.

Output → a system assessment report for every full run, same format every time

07

Compare against last time, then decide.

AI DOES

Diffs this report against past ones

New findings, regressions, and fixes

Coverage drift: tests skipped, weakened, or deleted

Model drift: the system changed and the model didn’t

HUMAN GATE

Judge the risk: part art, part skill, a lot of experience

Patch, redesign, or accept, with a named owner

Decide what goes back into stage 01

Output → decisions with owners, and the start of the next loop

Then start over. What kicks off a new loop?

- A design change
- A new component or vendor
- A new attack technique
- A finding in production
- The calendar: daily or weekly

Each loop inherits the last one’s tests, so coverage only grows.

Follow one requirement around the loop.

Stage	What happens
02 Analyze	REQ 3.2: session tokens die at logout. Human approves.
03 Plan	Test 3.2.1: replay a captured token after logout. High priority.
04 Write	AI writes a replay harness. Human re-plants the bug; the test catches it.
05 Run	Runs in every weekly full run. Green for months.
05 Run	Session service refactor is a material change. Full run triggers; 3.2.1 goes red.
06–07 Compare	Diff flags a regression vs. last release. Human blocks the release, names an owner.

Where this will burn you, and why the gates exist.

Made-up architecture

The model looks right and isn't. Check it against reality.

Tests that test nothing

Green forever. Plant the bug and prove they fail.

AI sprawl

AI tools want to be helpful, so they keep generating findings, tests, and agents. Prune and cap.

Drifting results

Version the prompts and models, so a diff means the system changed.

A map for attackers

The system model is sensitive. Protect it wherever it's processed.

Rubber stamps

A gate nobody reads is no gate. Humans actually sign.

What you get for the trouble.

	Annual pentest	AI-enabled program
Scope	The hosts we could hit	The whole system, as designed and built
Cadence	Once a year	Light checks per PR; full runs on material change or schedule
Design flaws	Rarely	First-class findings
Coverage	Implied	Stated, test by test
History	Last year’s PDF	A diff, every full run
Expert time	Running tools	Judgment

You don’t have to build all seven stages at once.

01

Write down the system. Even one page.

02

Have AI draft the requirements. Argue with it.

03

Turn your top ten into numbered tests in CI.

04

Diff each full run against the last. Then grow it.

Stuff worth stealing from.

DESIGN AND ARCHITECTURE

McGraw, *Software Security: Building Security In* (2006)

IEEE CSD, *Avoiding the Top 10 Software Security Design Flaws* (2014)

Shostack, *Threat Modeling* (2014)

Threat Modeling Manifesto (2020)

REQUIREMENTS AND QUALITY

NIST SP 800-160 Vol. 1

SEI ATAM quality attribute scenarios

ISO/IEC 25010

OWASP ASVS

MITRE CWE

Thanks for your time.

I hope this gets you thinking more broadly about how you assess the security of your systems.

Bruce Potter · gdead@shmoo.com · @gdead